

UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO - UERJ

Felipe Valentim da Silva

**CERTIFY: UM SISTEMA DESACOPLADO E DISTRIBUÍDO
PARA GESTÃO DE EVENTOS E CREDENCIAMENTO
INTELIGENTE**

RIO DE JANEIRO

2026

Felipe Valentim da Silva

CERTIFY: UM SISTEMA DESACOPLADO E DISTRIBUÍDO
PARA GESTÃO DE EVENTOS E CREDENCIAMENTO
INTELIGENTE

**Monografia apresentada ao curso de
Ciência da Computação da Univer-
sidade do Estado do Rio de Janeiro,
como requisito parcial para obtenção
do grau de Bacharel em Ciência da
Computação.**

Orientadora: Profa. Adriana Aparicio Sicsú
Ayres do Nascimento

Coorientador: Prof. Carlos Augusto Sicsú
Ayres do Nascimento

Rio de Janeiro, 2026

UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO

FELIPE VALENTIM DA SILVA

Esta Monografia foi julgada adequada para a obtenção do título de Bacharel em
Ciência da Computação, sendo aprovada em sua forma final pela banca examinadora:

Orientadora: Profa. Adriana Aparicio Sicsú
Ayres do Nascimento
Universidade do Estado do Rio de Janeiro -
UERJ

Coorientador: Prof. Carlos Augusto Sicsú
Ayres do Nascimento
Universidade do Estado do Rio de Janeiro -
UERJ

Membro da Banca 1
Instituição

Membro da Banca 2
Instituição

Rio de Janeiro, [DIA] de [MÊS] de 2026

Agradecimentos

Agradeço primeiramente a Deus pela força e determinação.

Aos meus orientadores, Profa. Adriana Aparicio Sicsú Ayres do Nascimento e Prof. Carlos Augusto Sicsú Ayres do Nascimento, pelo direcionamento técnico brilhante, pelo tempo investido em reuniões e pelas correções pontuais que elevaram o nível desta monografia e da arquitetura do projeto.

À Universidade do Estado do Rio de Janeiro (UERJ), por toda a infraestrutura acadêmica, pelo corpo docente formidável e pelos conhecimentos sólidos transmitidos ao longo dos anos de bacharelado.

Aos meus colegas de curso e amigos que apoiaram este projeto direta ou indiretamente, seja testando o aplicativo ou ajudando na revisão de código.

À minha família, pelo apoio incondicional e amor em todas as etapas da minha vida acadêmica.

Resumo

A gestão eficiente de eventos acadêmicos e corporativos apresenta desafios logísticos significativos, desde o controle rigoroso de inscrições até a emissão confiável de certificados. Este trabalho detalha o desenvolvimento do Certify, um sistema completo de credenciamento e gestão de eventos concebido para solucionar esses problemas sistêmicos. A plataforma adota uma arquitetura distribuída, estruturada através de uma API RESTful em .NET 6 fundamentada na *Onion Architecture*, e um aplicativo móvel cross-platform desenvolvido em React Native. O controle de segurança é assegurado pelo uso de tokens JWT stateless. Como diferenciais operacionais, o sistema utiliza o padrão EAV (Entity-Attribute-Value) para permitir a criação de formulários dinâmicos de inscrição, além de incorporar uma funcionalidade nativa de auto-checkin presencial instantâneo por meio da leitura de QR Codes criptografados com hashes. O fluxo da aplicação culmina na automação do motor documental, que utiliza algoritmos em C# para substituir identificadores fixos e dinâmicos em arquivos do Microsoft Word, convertendo-os em lote para PDF e distribuindo certificados em tempo real por e-mail. Os resultados indicam que o Certify mitiga os principais gargalos dos eventos modernos, eliminando o contato físico em bilheterias e o trabalho manual excessivo dos organizadores.

Palavras-chave: Gestão de Eventos; React Native; .NET 6; Onion Architecture; QR Code; Automação de Certificados.

Abstract

Efficient management of academic and corporate events presents significant logistical challenges, ranging from the rigorous control of registrations to the reliable issuance of certificates. This work details the development of Certify, a complete accreditation and event management system designed to solve these systemic problems. The platform adopts a distributed architecture, structured through a RESTful API in .NET 6 based on Onion Architecture, and a cross-platform mobile application developed in React Native. Security control is ensured by the use of stateless JWT tokens. As operational differentials, the system uses the EAV (Entity-Attribute-Value) pattern to allow the creation of dynamic registration forms, in addition to incorporating a native function for instant physical self-check-in through the reading of QR Codes encrypted with hashes. The application flow culminates in the automation of the document engine, which uses C# algorithms to replace fixed and dynamic identifiers in Microsoft Word files, converting them in batches to PDF and distributing certificates in real time via email. The results indicate that Certify mitigates the main bottlenecks of modern events, eliminating physical contact at box offices and the excessive manual labor of organizers.

Keywords: Event Management; React Native; .NET 6; Onion Architecture; QR Code; Certificate Automation.

Lista de ilustrações

Figura 1 – Diagrama de Casos de Uso do Sistema Certify	29
Figura 2 – Diagrama de Classes do Domínio	34
Figura 3 – Diagrama de Sequência: Realizar Login	35
Figura 4 – Diagrama de Sequência: Check-in via QR Code (Painel Admin)	35
Figura 5 – Diagrama de Sequência: Envio de Certificados em Lote	36
Figura 6 – Diagrama de Sequência: Cadastro de Evento	36
Figura 7 – Diagrama de Sequência: Gerenciamento de Campos Dinâmicos	37
Figura 8 – Diagrama de Sequência: Cadastro Manual de Convidado	37
Figura 9 – Diagrama de Sequência: Auto-cadastro Público de Convidado	38
Figura 10 – Diagrama de Sequência: Check-in em Lote	38
Figura 11 – Diagrama de Sequência: Upload de Template .docx	39
Figura 12 – Arquitetura Distribuída: Cliente, Servidor e Banco de Dados	39
Figura 13 – Modelo de Entidade Relacional (MER)	42
Figura 14 – Estrutura do Componente Evento (Onion Architecture)	45
Figura 15 – Fluxo da Camada de Frontend (App Mobile)	46
Figura 16 – Tela de Login (<code>login.js</code>)	48
Figura 17 – Tela de Criar Conta (<code>register.js</code>)	49
Figura 18 – Tela Inicial — Lista de Eventos (<code>home.js</code>)	49
Figura 19 – Criação de Novo Evento (<code>newEvent.js</code>)	50
Figura 20 – Aba de Informações do Evento (<code>eventTab.js</code>)	50
Figura 21 – Leitor de QR Code (<code>eventScanner.js</code>)	51
Figura 22 – Gerenciamento de Campos Dinâmicos (<code>eventFields.js</code>)	51
Figura 23 – Criação de Novo Campo Dinâmico (<code>newField.js</code>)	52
Figura 24 – Lista de Convidados e Seleção em Massa (<code>guestsTab.js</code>)	52
Figura 25 – Cadastro Manual de Convidado (<code>newGuest.js</code>)	53
Figura 26 – Detalhes do Convidado (<code>guest.js</code>)	54
Figura 27 – Aba de Certificado e Template (<code>templateTab.js</code>)	54

Lista de tabelas

Tabela 1	–	CDU001 – Criar Conta	30
Tabela 2	–	CDU002 – Realizar Login	30
Tabela 3	–	CDU004 – Criar Evento	31
Tabela 4	–	CDU007 – Cadastrar Convidado Manualmente	31
Tabela 5	–	CDU009 – Check-in via QR Code (Painel Admin)	32
Tabela 6	–	CDU012 – Gerar e Enviar Certificados em Lote	33
Tabela 7	–	Mapeamento Completo de Endpoints da API Certify	44

Lista de abreviaturas e siglas

API	Application Programming Interface
DTO	Data Transfer Object
EAV	Entity-Attribute-Value
EF Core	Entity Framework Core
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
PDF	Portable Document Format
QR Code	Quick Response Code
REST	Representational State Transfer
RN	React Native
SGBD	Sistema Gerenciador de Banco de Dados
SoC	Separation of Concerns
UUID	Universally Unique Identifier

Sumário

1	INTRODUÇÃO	19
1.1	Motivação	19
1.2	Justificativa	20
1.3	Objetivos	20
1.3.1	Objetivo Geral	20
1.3.2	Objetivos Específicos	20
1.4	Estrutura do Trabalho	21
2	CONTEXTUALIZAÇÃO E DESAFIOS NA GESTÃO DE EVENTOS	23
2.1	O Paradigma Atual do Credenciamento	23
2.2	O Gargalo da Certificação Manual	23
2.3	A Proposta de Engenharia do Certify	23
3	FUNDAMENTAÇÃO TEÓRICA	25
3.1	.NET 6 e a Arquitetura em Cebola (Onion Architecture)	25
3.2	React Native, Expo e Redux	25
3.3	A Segurança Stateless com JWT	26
3.4	EAV: Entity-Attribute-Value Model	26
4	METODOLOGIA	27
4.1	Requisitos Funcionais	27
4.2	Requisitos Não Funcionais	28
4.3	Diagrama de Casos de Uso	29
4.4	Descrição dos Casos de Uso	30
4.5	Diagrama de Classes	33
4.6	Diagramas de Sequências	35
4.7	Visão Geral do Projeto do Sistema	39
4.8	Visão do Banco de Dados	40
4.8.1	SGBD	40
4.8.2	MER - Modelo de Entidade Relacional	42
4.9	Visão do Backend (API RESTful)	43
4.9.1	Tecnologias a serem usadas	43
4.9.2	Projeto do Backend (Onion Architecture)	43
4.9.3	Endpoints da API	44
4.9.4	Estrutura de Componentes da API	44
4.10	Visão do Frontend (App Mobile)	46

5	APRESENTAÇÃO DO SISTEMA	47
5.1	Tecnologias a serem usadas no Frontend	47
5.2	Projeto das Interfaces	48
6	CONSIDERAÇÕES FINAIS	55
6.1	Conclusão	55
6.2	Implementações Futuras	55

1 Introdução

O avanço tecnológico impulsionado pela computação móvel e pelas aplicações distribuídas alterou fundamentalmente o paradigma da gestão de informações organizacionais. Historicamente, a coordenação de grandes fluxos de dados presenciais dependia de estruturas rígidas, lentas e suscetíveis a falhas humanas consideráveis. Com o surgimento dos *smartphones* e o estabelecimento de redes de alta velocidade, processos que antes levavam horas agora podem ser concluídos em frações de segundo mediante leitura óptica ou transmissões assíncronas.

Neste cenário de evolução digital, a organização de eventos — sejam eles congressos acadêmicos, simpósios corporativos ou workshops informais — permanece como um nicho onde a transformação digital muitas vezes é implementada de forma fragmentada. Organizadores ainda enfrentam grandes barreiras logísticas, especialmente no momento de credenciar presencialmente os convidados e no encerramento do ciclo, que envolve a emissão segura e correta de milhares de diplomas ou certificados nominais (??).

O **Certify** foi idealizado para ser não apenas uma solução paliativa, mas um ecossistema definitivo que resolve a jornada completa de um evento. Ele centraliza a relação entre os administradores do evento e os seus convidados, orquestrando um grande volume de dados de forma assíncrona, robusta e livre de papel (*paperless*).

1.1 Motivação

A motivação original para o projeto surge da vivência acadêmica e profissional, onde é evidente o atraso logístico na entrada de eventos. Longas filas para assinar listas de presenças físicas ainda são corriqueiras. Além da demora no credenciamento, a extração dessas assinaturas de papel para planilhas digitais visando a criação de certificados é um processo maçante.

Muitas vezes, a necessidade de produzir certificados com dados além do nome (como matrícula, curso ou área de atuação) aumenta as chances de erros ortográficos ou troca de dados. Construir um motor computacional capaz de substituir todas essas etapas burocráticas por meio de leitura instantânea via QR Code e geração autônoma de PDFs no lado do servidor foi o grande desafio motivador para a idealização do Certify.

1.2 Justificativa

Atualmente, o mercado disponibiliza plataformas de gestão de eventos, porém elas esbarram em dois limitadores cruciais: altas taxas de comissionamento comercial e arquiteturas fechadas (muitas vezes dependentes de interfaces web lentas e não otimizadas para o dinamismo do dia do evento pelo celular).

Justifica-se, portanto, a criação de uma plataforma independente com uma arquitetura moderna nativamente focada em dispositivos móveis (*Mobile First*). Optou-se pela construção do frontend do aplicativo utilizando a biblioteca **React Native** para obter compilação cross-platform e uma API **.NET 6** que aplica a rigorosa *Onion Architecture* (?), provendo um sistema altamente testável, manutenível e preparado para lidar com operações pesadas (como conversão de streams de PDF em memória) sem prejudicar a interface do usuário no aparelho móvel.

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver e implementar uma plataforma de software distribuída (aplicativo móvel e API RESTful) nomeada "Certify", visando automatizar integralmente os processos operacionais de gestão de eventos, desde o cadastro flexível de convidados até a geração automatizada de certificados através de fluxos de substituição de dados.

1.3.2 Objetivos Específicos

Considerando as vertentes técnicas da engenharia de software necessárias para concretizar o projeto, os objetivos específicos englobam:

- Projetar uma API modular utilizando o framework **.NET 6**, isolando as regras de negócios da camada de infraestrutura sob as premissas da *Onion Architecture*.
- Desenvolver um banco de dados relacional e escalável fazendo uso do padrão *Soft Delete* (exclusão lógica) e chaves primárias universais (UUID).
- Implementar o padrão *Entity-Attribute-Value* (EAV) para permitir que os organizadores criem formulários de cadastro dinâmicos adaptáveis a qualquer evento.
- Incorporar um mecanismo de credenciamento utilizando decodificação de criptografia de QR Codes diretamente pelo hardware móvel.

- Criar um motor de processamento no backend que carregue *templates* de Word, substitua marcadores pelos dados do banco de dados, converta-os em PDF e os despache via correio eletrônico de forma automatizada.

1.4 Estrutura do Trabalho

Esta monografia está estruturada de maneira sequencial e construtiva, mapeando todo o processo de engenharia do Certify: No **Capítulo 2**, é analisado o panorama atual do problema, contextualizando a necessidade de inovação na gestão de eventos. No **Capítulo 3**, são expostos os referenciais teóricos das tecnologias de vanguarda escolhidas. O **Capítulo 4** compõe o núcleo técnico, demonstrando as regras de negócios, modelagem EAV, fluxogramas, e as estratégias de autenticação com segurança stateless. O **Capítulo 5** concentra-se na Engenharia de Interface, apresentando os resultados das telas, controles de gestos e navegação reativa. O documento se encerra no **Capítulo 6**, com a síntese dos resultados alcançados e possíveis avanços futuros.

2 Contextualização e Desafios na Gestão de Eventos

2.1 O Paradigma Atual do Credenciamento

A gestão de eventos contemporânea, particularmente em âmbitos acadêmicos e corporativos de médio porte, sofre de um hiato tecnológico. Enquanto áreas como o comércio eletrônico e a logística adotaram sistemas altamente integrados, o processo de recepção presencial de convidados ainda se apoia fortemente em metodologias analógicas.

Atualmente, é comum que a inscrição dos participantes ocorra por meios digitais (como formulários do *Google Forms*), mas o credenciamento no dia do evento retroceda para planilhas impressas em papel. Recepcionistas passam horas procurando manualmente nomes em listas para atestar a presença, um processo suscetível a filas quilométricas, desorganização física e perda de dados valiosos sobre a pontualidade ou taxa de desistência dos convidados.

2.2 O Gargalo da Certificação Manual

A falha sistêmica se estende para o pós-evento. Como o controle de presença foi feito em papel, organizadores são obrigados a digitar ou transcrever novamente os dados para softwares de edição de texto ou depender de sistemas de "Mala Direta" (como a integração entre Excel e Microsoft Word) para gerar os certificados. Esse processo consome horas de trabalho humano.

Além da lentidão, existe uma inflexibilidade enorme quando se trata de coletar e exibir informações variáveis. Por exemplo, se um evento precisa colocar o "Cargo" do participante corporativo no certificado, e o outro evento exige o número da "Matrícula" acadêmica do aluno, o fluxo estático de mala direta frequentemente se quebra. Erros de formatação, nomes cortados ou omissão de envio por e-mail geram desgaste na imagem da organização responsável pelo evento.

2.3 A Proposta de Engenharia do Certify

Em resposta a essas ineficiências latentes, o sistema Certify atua reconstruindo toda essa arquitetura sob a perspectiva da automação. Em vez de utilizar softwares segmentados (um para formulário, outro para lista em papel, outro para mala direta e outro provedor

de e-mail), o Certify engloba o ciclo de vida inteiro de uma "Unidade de Presença" no seu banco de dados.

A proposta do sistema não é apenas informatizar, mas aplicar Engenharia de Software no fluxo de trabalho:

- ****Inscrição Dinâmica:**** O uso de bancos relacionais com propriedades flexíveis (Padrão EAV) permite que qualquer formulário seja gerado de forma autônoma sem requerer alteração de tabelas físicas no banco de dados.
- ****Credenciamento Assíncrono e Seguro:**** A adoção de leitura óptica (câmeras de celulares processando algoritmos criptográficos em código QR) retira o gargalo humano da validação. A API .NET analisa os limites temporais (Data e Horário) definidos pelo administrador e processa transações na casa dos milissegundos.
- ****Automação Pós-Evento:**** O sistema encerra o fluxo atuando como um motor de geração de documentos na memória, evitando sobrecarga de discos rígidos do servidor, e aciona o protocolo SMTP para entregar PDFs gerados diretamente na caixa de entrada do participante, exigindo zero esforço de clique do organizador após a configuração inicial do template Word.

3 Fundamentação Teórica

O desenvolvimento do Certify embasou-se em literaturas consolidadas de engenharia de software e na aplicação pragmática de **frameworks** de ponta. Este capítulo discorre sobre os conceitos teóricos das tecnologias empregadas.

3.1 .NET 6 e a Arquitetura em Cebola (Onion Architecture)

O .NET 6 é uma plataforma de desenvolvimento transversal de código aberto mantida pela Microsoft. A escolha dessa tecnologia justifica-se pela sua extrema velocidade na compilação **Just-in-Time** e excelente suporte à criação de APIs RESTful utilizando a linguagem C# (??).

O diferencial arquitetural da API do Certify, entretanto, está na adoção do padrão **Onion Architecture** (Arquitetura Cebola). Esta metodologia defende que o coração da aplicação é o Domínio (entidades puras e sem dependência de bibliotecas externas). Em volta do Domínio orbitam as camadas de Serviço (Regras de Negócio) e Repositório (Persistência). A parte mais externa é a camada de Apresentação (os **Controllers** da API). Ao utilizar o Princípio da Inversão de Dependência (SOLID), o núcleo da aplicação torna-se altamente testável e livre de acoplamento direto com o Banco de Dados.

3.2 React Native, Expo e Redux

O React Native é um *framework* desenvolvido pela Meta Platforms que permite a escrita de aplicativos móveis utilizando a sintaxe JavaScript/TypeScript (??). Ao invés de usar WebViews nativas, o framework invoca APIs nativas de interface do iOS e do Android de forma reativa, o que propicia uma performance análoga à do desenvolvimento nativo (Java/Kotlin ou Swift), mas com uma base de código unificada.

A integração com o ***Expo (SDK 51)*** (??) foi crucial para acelerar a engenharia do projeto, dado que ele fornece módulos robustos para acesso ao hardware do celular (como a câmera em **expo-camera**, fundamental para a leitura do QR Code, e o **expo-secure-store** para a criptografia do token de segurança local).

Adicionalmente, o ***Redux Toolkit*** implementa o padrão **Flux** para gerenciar os estados da aplicação. Ele centraliza informações críticas, como as notificações flutuantes (Snackbar) e os dados do usuário autenticado, disparando atualizações que reconstroem as telas do frontend em milissegundos de forma imutável e escalável.

3.3 A Segurança Stateless com JWT

O JSON Web Token (JWT) é um padrão aberto (RFC 7519) que define um método compacto e autônomo para transmitir informações seguras entre partes como um objeto JSON. A premissa do JWT é a arquitetura **Stateless** (sem estado), um requisito fundamental para sistemas escaláveis em nuvem.

No Certify, ao invés do servidor armazenar as sessões na memória (o que limitaria o crescimento horizontal do sistema), o servidor valida o hash de senha (BCrypt) e assina digitalmente um JWT contendo as **Claims** do usuário utilizando a criptografia simétrica HMAC-SHA256. A segurança não se dá por esconder os dados (já que o payload em Base64 é visível), mas sim pela impossibilidade de um atacante forjar a assinatura sem a Chave Secreta mantida na infraestrutura do backend.

3.4 EAV: Entity-Attribute-Value Model

Bancos de dados relacionais sofrem com a rigidez de colunas. Se o sistema permitisse que o organizador gerasse novos campos customizados criando colunas dinâmicas em tabelas físicas na linguagem SQL, o risco de instabilidade (**Schema Locking**) seria enorme.

A solução matemática e de arquitetura encontrada na literatura é o modelo Entity-Attribute-Value (EAV). Neste padrão as informações são particionadas horizontalmente: - A Entidade (**Entity**): o identificador do evento. - O Atributo (**Attribute**): o nome ou tipo do campo (Ex: "Matrícula"). - O Valor (**Value**): a resposta literal concedida pelo convidado.

Esta estratégia viabilizou a premissa fundamental do Certify de oferecer flexibilidade máxima sem corromper as restrições da modelagem relacional formal.

4 Metodologia

Neste capítulo são apresentadas as especificações de requisitos levantadas diretamente a partir do código-fonte, a modelagem UML do sistema e a estrutura arquitetural completa do backend e banco de dados.

4.1 Requisitos Funcionais

Os requisitos funcionais foram extraídos da análise dos *Controllers*, *Services* e entidades do projeto:

- **[RF001] Criar Conta:** O organizador deve poder se registrar na plataforma informando nome, e-mail e senha.
- **[RF002] Autenticar no Sistema:** O sistema deve validar as credenciais do organizador, comparando o hash BCrypt da senha armazenada, e retornar um token JWT.
- **[RF003] Listar e Filtrar Eventos:** O sistema deve listar apenas os eventos pertencentes ao organizador autenticado, extraíndo o identificador do usuário diretamente do token JWT.
- **[RF004] Criar Evento:** O sistema deve permitir a criação de eventos com nome, tipo, data, horários de início e fim, capacidade máxima (*Pax*) e foto de banner. A data de início não pode ser anterior ao momento da criação.
- **[RF005] Excluir Evento:** O sistema deve excluir logicamente (*Soft Delete*) um evento, preenchendo o campo *DeletedDate* sem apagar o registro do banco.
- **[RF006] Gerenciar Campos Dinâmicos (EAV):** O organizador deve poder criar campos personalizados de formulário (Texto, Número ou Data), definir obrigatoriedade e reordenar os campos via operação de lote.
- **[RF007] Cadastrar Convidados Manualmente:** O organizador deve poder cadastrar convidados diretamente pelo painel, com validação de e-mail (expressão regular), tipo de convidado e campos dinâmicos obrigatórios do evento.
- **[RF008] Auto-cadastro Público de Convidados:** O sistema deve disponibilizar rotas públicas (*AllowAnonymous*) para que o próprio participante preencha o formulário de inscrição, sendo bloqueado caso o horário de início do evento já tenha passado ou o limite de vagas (*Pax*) tenha sido atingido.

- **[RF009] Check-in via QR Code (Painel Administrativo):** O aplicativo deve capturar o QR Code do participante via câmera, extrair o UUID do convidado e acionar o endpoint de check-in autenticado.
- **[RF010] Check-in Automático (Rota Pública):** O sistema deve disponibilizar uma rota pública decodificada de hash para que o convidado realize seu próprio check-in, respeitando o modo do evento (`CheckinEnabled`).
- **[RF011] Gerenciar Check-in em Lote:** O organizador deve poder aprovar ou reverter check-ins de múltiplos convidados simultaneamente via seleção em massa na interface.
- **[RF012] Upload de Template de Certificado:** O sistema deve aceitar arquivos Word (`.doc/.docx`) de até 10 MB, validar a presença do placeholder `{Nome}` e gerar uma pré-visualização em PDF.
- **[RF013] Enviar Convites por E-mail:** O sistema deve disparar e-mails com HTML personalizado e link do QR Code de auto-checkin para os convidados selecionados que ainda não fizeram check-in.
- **[RF014] Gerar e Enviar Certificados em Lote por E-mail:** O sistema deve substituir os *placeholders* fixos e dinâmicos (EAV) no template Word, converter para PDF em memória via *MemoryStream* e enviar os arquivos por e-mail para todos os convidados com check-in confirmado.
- **[RF015] Download de Certificados em ZIP:** O sistema deve gerar um arquivo compactado (`.zip`) contendo os PDFs individuais de todos os convidados presentes e disponibilizá-lo para download.

4.2 Requisitos Não Funcionais

- **[RNF001] Autenticação JWT Stateless:** O servidor não armazena sessões. A validade e a identidade do usuário são verificadas a cada requisição pelo *middleware* `[Authorize]`, que valida a assinatura HMAC-SHA256 do token.
- **[RNF002] Chaves Primárias UUID (Guid):** Todas as entidades herdam de `EntityBase`, que define o campo `Id` como `Guid`. O identificador é gerado pela camada de aplicação (`Guid.NewGuid()`) antes de qualquer persistência.
- **[RNF003] Soft Delete (Exclusão Lógica):** A classe `AuditableEntity` define os campos `CreatedDate` (preenchido automaticamente com `DateTime.UtcNow` no construtor) e `DeletedDate` (nulo por padrão). Exclusões apenas preenchem `DeletedDate`; o registro permanece intacto no banco.

- [RNF004] **Onion Architecture:** O projeto é dividido em projetos isolados: **Domain** (entidades e interfaces puras, sem dependências externas), **Services** (regras de negócio), **Repository** (acesso a dados via EF Core), **Infra** (implementações de serviços externos) e **API** (camada de apresentação).
- [RNF005] **Armazenamento Seguro no Mobile:** O token JWT é persistido no dispositivo via **expo-secure-store**, que delega a criptografia ao *Keychain* (iOS) ou *KeyStore* (Android), sendo inacessível a outros aplicativos.
- [RNF006] **Processamento de Documentos em Memória:** A geração de PDF é realizada integralmente via **MemoryStream**, sem criação de arquivos temporários em disco, prevenindo acúmulo de lixo no servidor.
- [RNF007] **Fonte Tipográfica Consistente:** O aplicativo carrega a família tipográfica *Poppins* (Regular, Bold, Black e ExtraBold) de forma assíncrona na inicialização via **expo-font**, garantindo consistência visual entre iOS e Android.

4.3 Diagrama de Casos de Uso

A Figura 1 ilustra as interações dos dois atores do sistema com as funcionalidades mapeadas a partir dos *controllers*. O **Organizador** acessa as rotas protegidas pelo [Authorize], enquanto o **Convidado** interage exclusivamente com o **FormController** ([AllowAnonymous]).

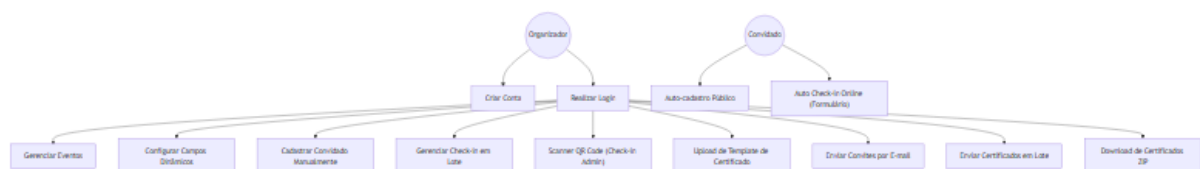


Figura 1 – Diagrama de Casos de Uso do Sistema Certify

4.4 Descrição dos Casos de Uso

Tabela 1 – CDU001 – Criar Conta

Identificador	CDU001 – Criar Conta
Objetivo	Permitir que um novo organizador crie suas credenciais de acesso ao sistema.
Ator principal	Organizador
Pré-condições	Nenhuma.
Pós-condições	Registro inserido na tabela <code>UserProfile</code> com senha armazenada em hash BCrypt.
Estrutura de Dados	Nome, E-mail, Senha.
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Acessa a tela de Registro e preenche os campos Nome, E-mail e Senha.	2. O app valida os campos localmente (formato do e-mail e comprimento mínimo da senha).
3. Submete o formulário.	4. A API aplica o hash BCrypt na senha e persiste o objeto <code>UserProfile</code> no banco. Retorna HTTP 200 OK.

Tabela 2 – CDU002 – Realizar Login

Identificador	CDU002 – Realizar Login
Objetivo	Autenticar o organizador e iniciar a sessão local no dispositivo.
Ator principal	Organizador
Pré-condições	Conta ativa cadastrada no sistema.
Pós-condições	Token JWT salvo no <code>expo-secure-store</code> ; Redux atualizado com <code>signIn</code> ; usuário redirecionado para a tela Home.
Estrutura de Dados	E-mail e Senha.
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Informa E-mail e Senha e toca em "Entrar".	2. O app valida o formato do e-mail via regex e o comprimento mínimo da senha.
3. Aguarda resposta.	4. A API consulta o <code>UserProfile</code> , compara o hash da senha com BCrypt e, se válido, assina um JWT com HMAC-SHA256 e retorna o token.
	5. O app salva o token no <code>expo-secure-store</code> , despacha <code>signIn</code> para a Redux Store e navega para a Home.
Fluxo Alternativo	
2a. Se o e-mail ou senha forem inválidos, o app exibe um <code>CustomSnackBar</code> de erro sem realizar a chamada à API. 4a. Se as credenciais não corresponderem, a API retorna HTTP 400 e o app exibe o snackbar "E-mail ou senha incorretos".	

Tabela 3 – CDU004 – Criar Evento

Identificador	CDU004 – Criar Evento
Objetivo	Registrar um novo evento vinculado ao organizador autenticado.
Ator principal	Organizador
Pré-condições	Usuário autenticado. Tipos de evento disponíveis (EventType).
Pós-condições	Evento persistido na tabela Event com o UserId extraído do JWT.
Estrutura de Dados	Nome, Tipo de Evento, Data, Horário Inicial, Horário Final, Capacidade (Pax), Foto.
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Acessa o formulário de novo evento e preenche os campos.	2. O app utiliza DateTimePicker nativo para data/hora e expo-image-picker para a foto.
3. Confirma a criação.	4. A API valida: nome não vazio, data/hora inicial não no passado, horário inicial menor que o final, Pax maior que zero. Persiste o evento e retorna o UUID gerado.
Fluxo Alternativo	
4a. Se o StartTime for maior ou igual ao EndTime , a API lança BusinessException : "Horário inicial não pode ser maior que horário final.". 4b. Se a data e hora inicial já tiverem passado, lança: "A data e horário inicial já passaram."	

Tabela 4 – CDU007 – Cadastrar Convidado Manualmente

Identificador	CDU007 – Cadastrar Convidado Manualmente
Objetivo	Registrar um participante em um evento pelo painel administrativo do aplicativo.
Ator principal	Organizador
Pré-condições	Evento ativo existente.
Pós-condições	Convidado inserido em Guest ; respostas EAV em EventFieldValue ; convite enviado por e-mail automaticamente.
Estrutura de Dados	Nome, E-mail, Tipo de Convidado, Foto (opcional), Campos Dinâmicos EAV.
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Preenche nome, e-mail, tipo e campos extras; envia.	2. A API valida: nome e e-mail obrigatórios, e-mail por regex, tipo de convidado definido, campos EAV obrigatórios preenchidos e com tipo correto (Number/Date), limite de Pax não atingido, e-mail não duplicado no evento.
	3. Se tudo válido, insere Guest e os EventFieldValue ; dispara convite por e-mail com link QR Code.
Fluxos Alternativos	
2a. E-mail duplicado no evento: "Email já cadastrado para este evento.". 2b. Limite de vagas atingido: "Limite de convidados atingido para este evento.". 2c. Campo EAV obrigatório vazio: "{NomeDoCampo} é obrigatório.".	

Tabela 5 – CDU009 – Check-in via QR Code (Painel Admin)

Identificador	CDU009 – Check-in via QR Code
Objetivo	Registrar presença de um convidado por leitura óptica do QR Code.
Ator principal	Organizador
Pré-condições	Convidado cadastrado no evento; evento com <code>CheckinEnabled</code> compatível.
Pós-condições	Campo <code>CheckinDate</code> do <code>Guest</code> preenchido com o horário de Brasília.
Estrutura de Dados	UUID do convidado (embarcado no QR Code em texto puro nesta rota admin).
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Aponta a câmera para o QR Code do participante na aba "Scanner".	2. O <code>CameraView</code> (expo-camera) detecta o código e valida se o dado está dentro dos limites do visor (<code>scannerBounds</code>).
3. Aguarda confirmação.	4. O app localiza o convidado pela lista local (sem round-trip ao servidor); navega automaticamente para a tela de detalhes do convidado (<code>guest.js</code>) se encontrado.
	5. A API (<code>PUT /Guest/Checkin/{id}</code>) valida se o check-in já foi realizado e grava <code>CheckinDate = DateTime.UtcNow.ConvertToBrazilTime()</code> .
Fluxos Alternativos	
2a. QR Code fora dos limites do visor: leitura ignorada. 2b. Convidado não pertence ao evento: snackbar "QRCode inválido para este evento". 5a. Check-in já realizado: API retorna <code>HTTP 409 Conflict</code> ; app exibe feedback visual.	

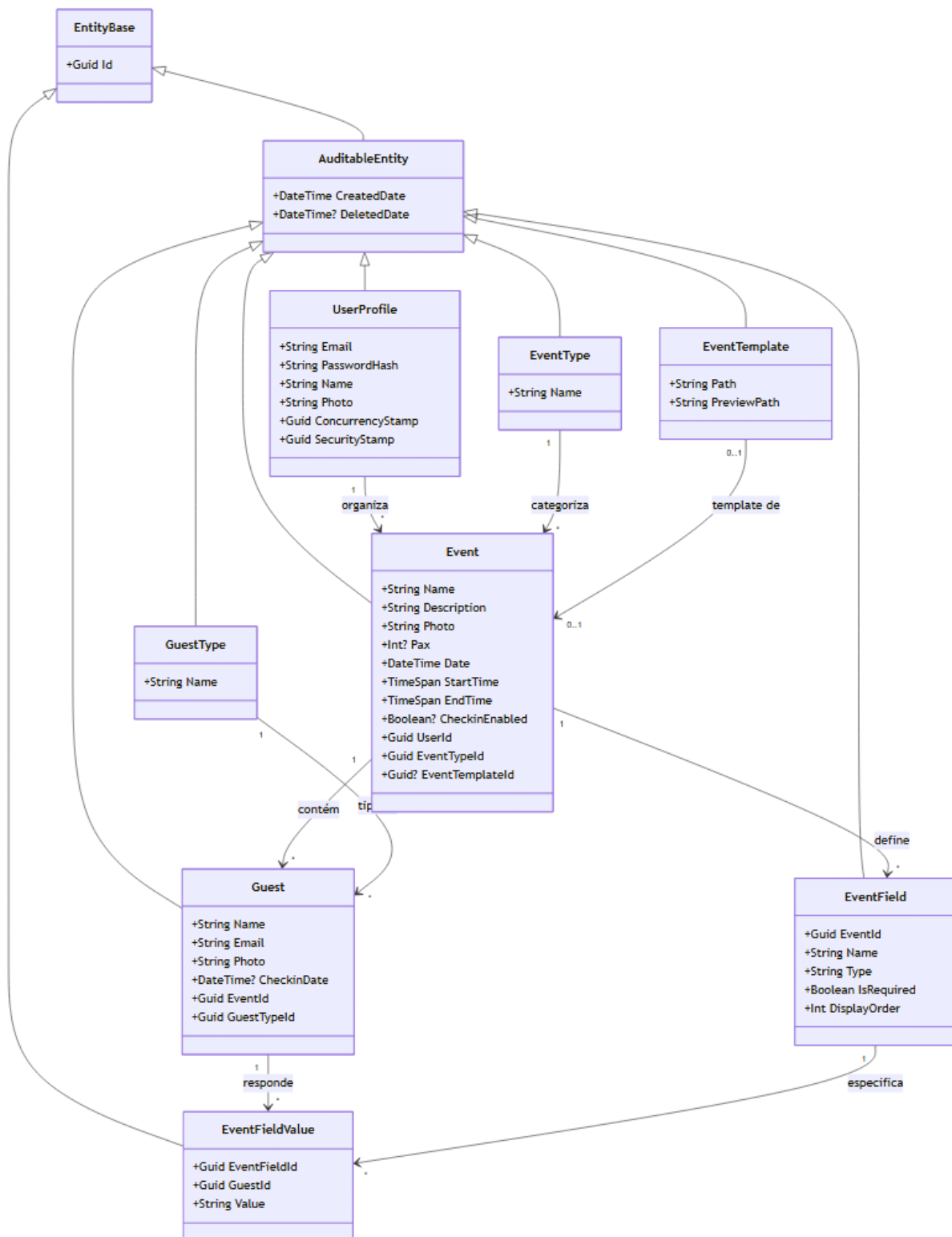
Tabela 6 – CDU012 – Gerar e Enviar Certificados em Lote

Identificador	CDU012 – Gerar e Enviar Certificados em Lote
Objetivo	Substituir os <i>placeholders</i> do template Word com dados do banco, converter para PDF e enviar por e-mail.
Ator principal	Organizador
Pré-condições	Evento com template .docx configurado; ao menos um convidado com CheckinDate preenchido.
Pós-condições	Certificados PDF individuais despachados para os e-mails dos convidados presentes.
Estrutura de Dados	Placeholders fixos ({Nome}, {Data}, {Evento}, {Tipo Convidado}, {Tipo Evento}, {Horário Inicial}, {Horário Final}) e dinâmicos (campos EAV).
Fluxo Principal	
Ações do Usuário	Ações do Sistema
1. Seleciona convidados presentes e aciona o envio via menu de contexto.	2. A API filtra convidados com CheckinDate != null pertencentes ao lote selecionado.
	3. Para cada convidado: carrega o .docx via DocX.Load() ; aplica ReplaceText() com StringReplaceTextOptions para campos fixos; itera FieldsValues para campos EAV dinâmicos; salva em MemoryStream .
	4. Converte o stream via Spire.Doc para PDF; monta o MailMessageDTO com o PDF como anexo e HTML do template grateful.html .
	5. Despacha a lista de e-mails em lote via MailService.SendMailCheckfyAsync() .
Fluxos Alternativos	
Pré-condição não atendida: API lança NotFoundException com mensagem "Evento não possui template." ou "Evento não possui convidados com checkin.".	

4.5 Diagrama de Classes

O diagrama abaixo representa a hierarquia real de entidades da camada **Domain**, conforme implementada nos arquivos **.cs** do projeto. Todas as entidades auditáveis herdam de **AuditableEntity**, que por sua vez herda de **EntityBase** (que define o **Guid Id**).

Figura 2 – Diagrama de Classes do Domínio



4.6 Diagramas de Sequências

Figura 3 – Diagrama de Sequência: Realizar Login

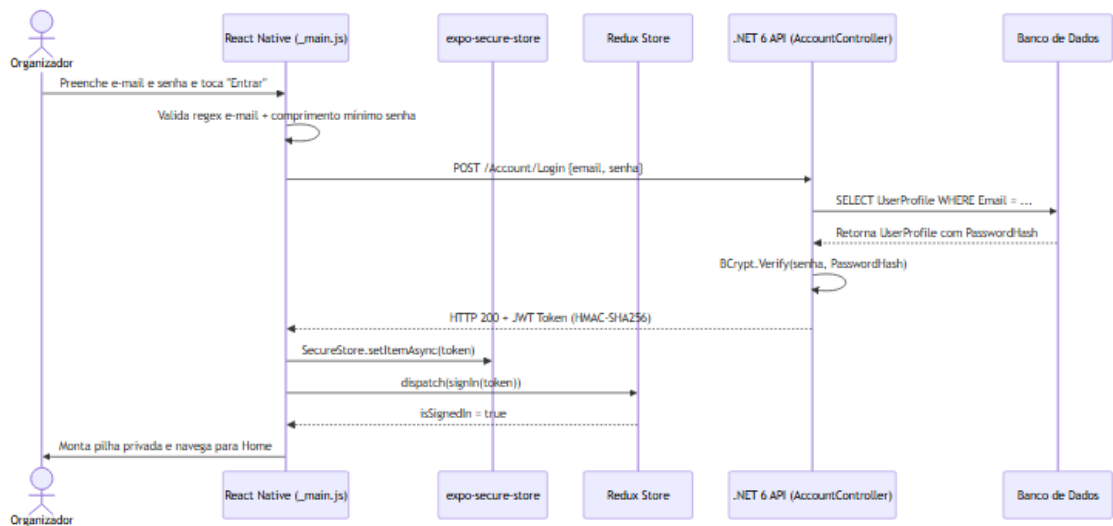


Figura 4 – Diagrama de Sequência: Check-in via QR Code (Painel Admin)

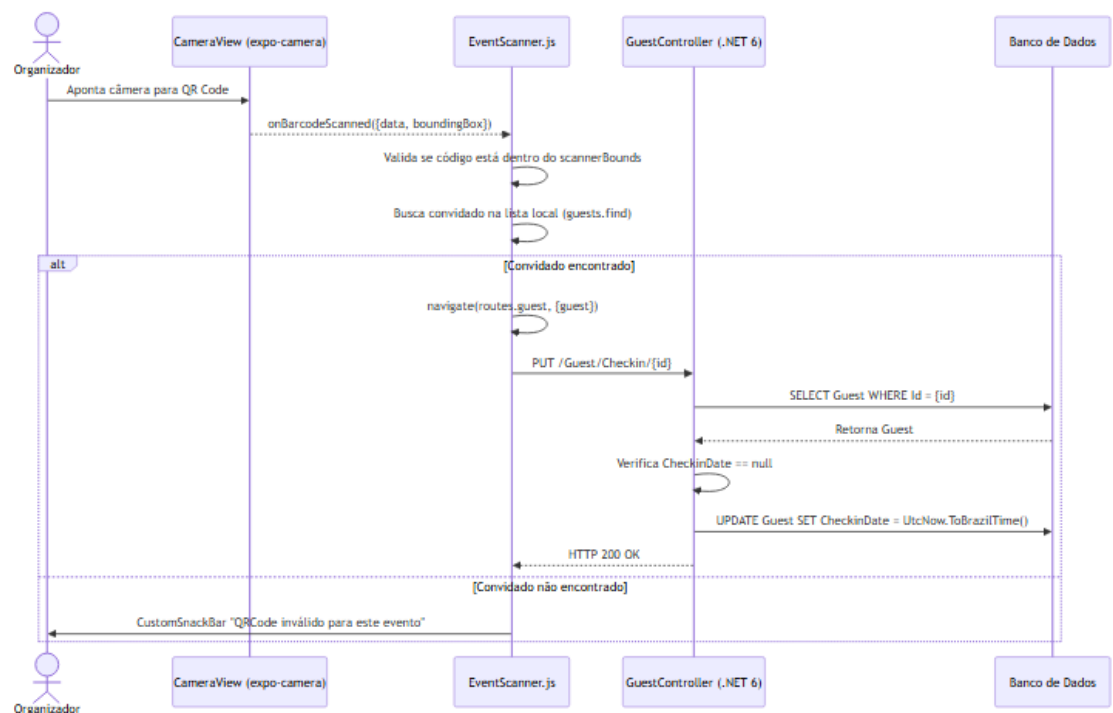


Figura 5 – Diagrama de Sequência: Envio de Certificados em Lote

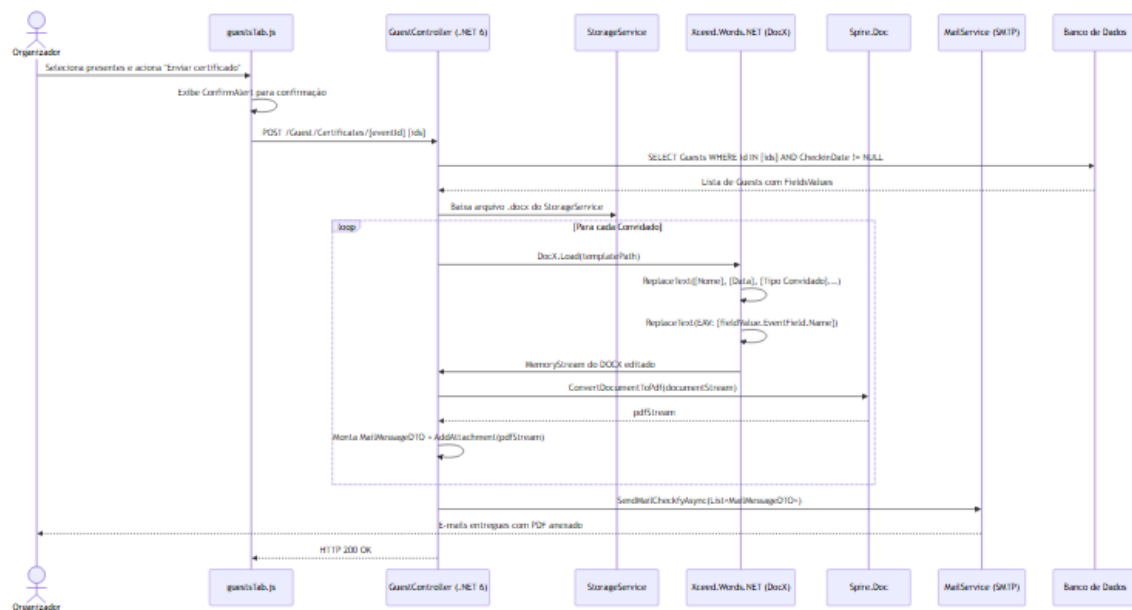


Figura 6 – Diagrama de Sequência: Cadastro de Evento

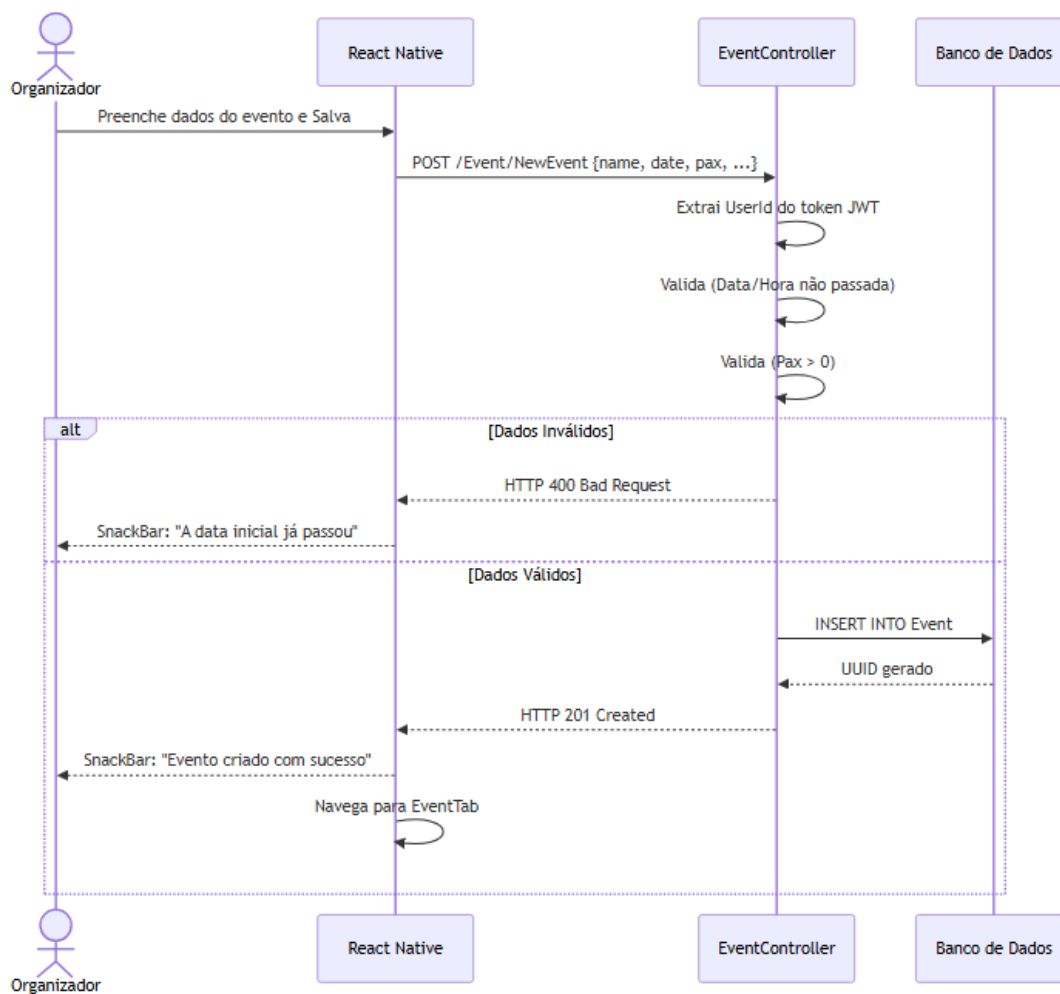


Figura 7 – Diagrama de Sequência: Gerenciamento de Campos Dinâmicos

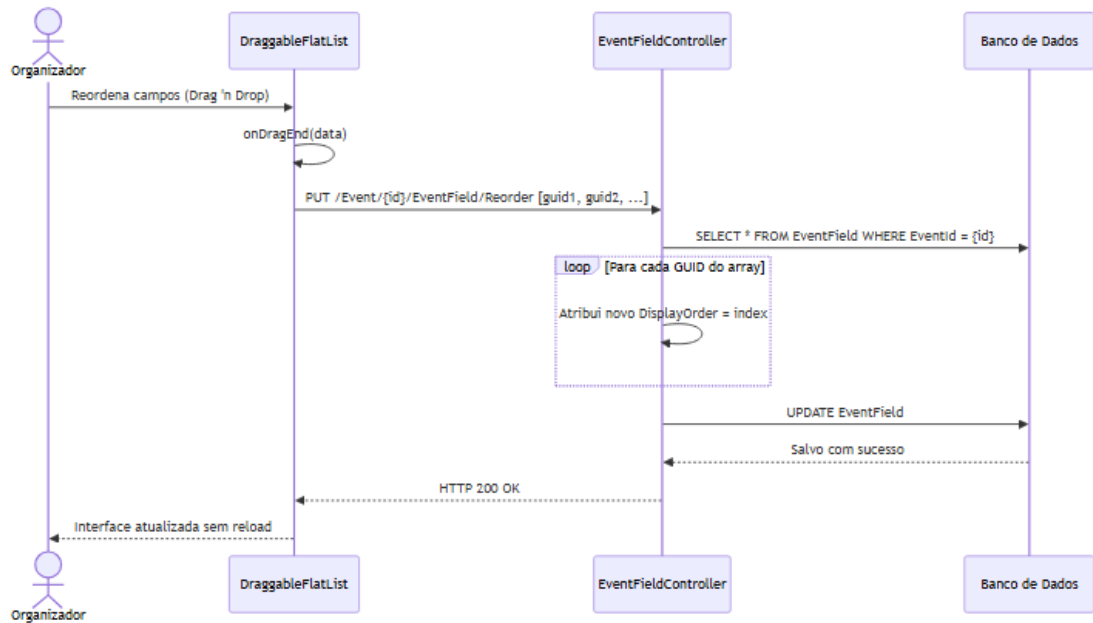


Figura 8 – Diagrama de Sequência: Cadastro Manual de Convidado

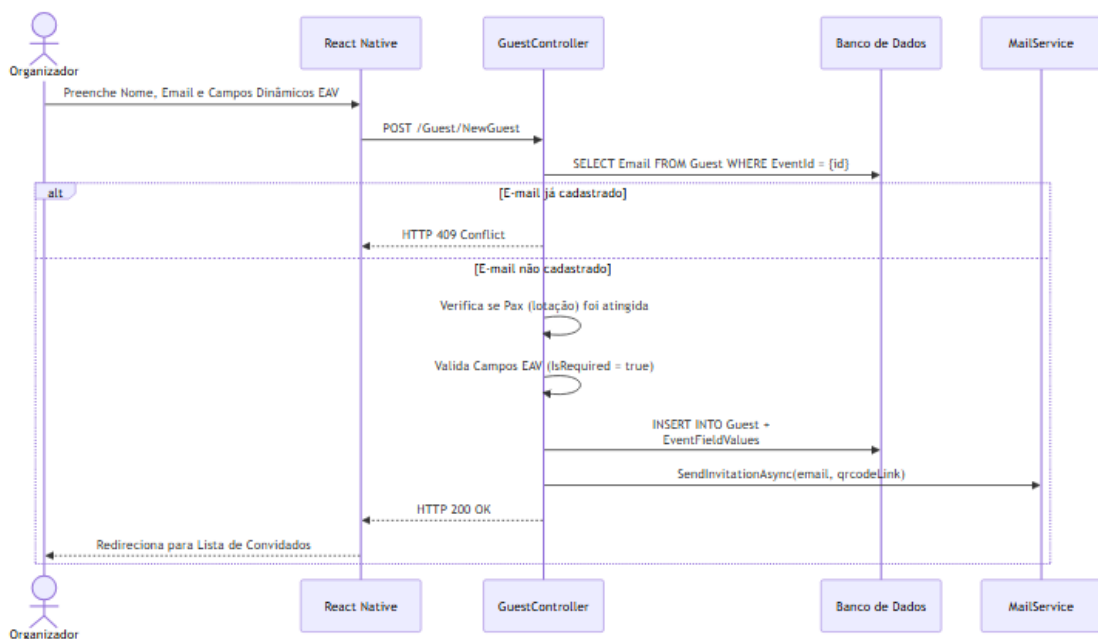


Figura 9 – Diagrama de Sequência: Auto-cadastro Público de Convidado

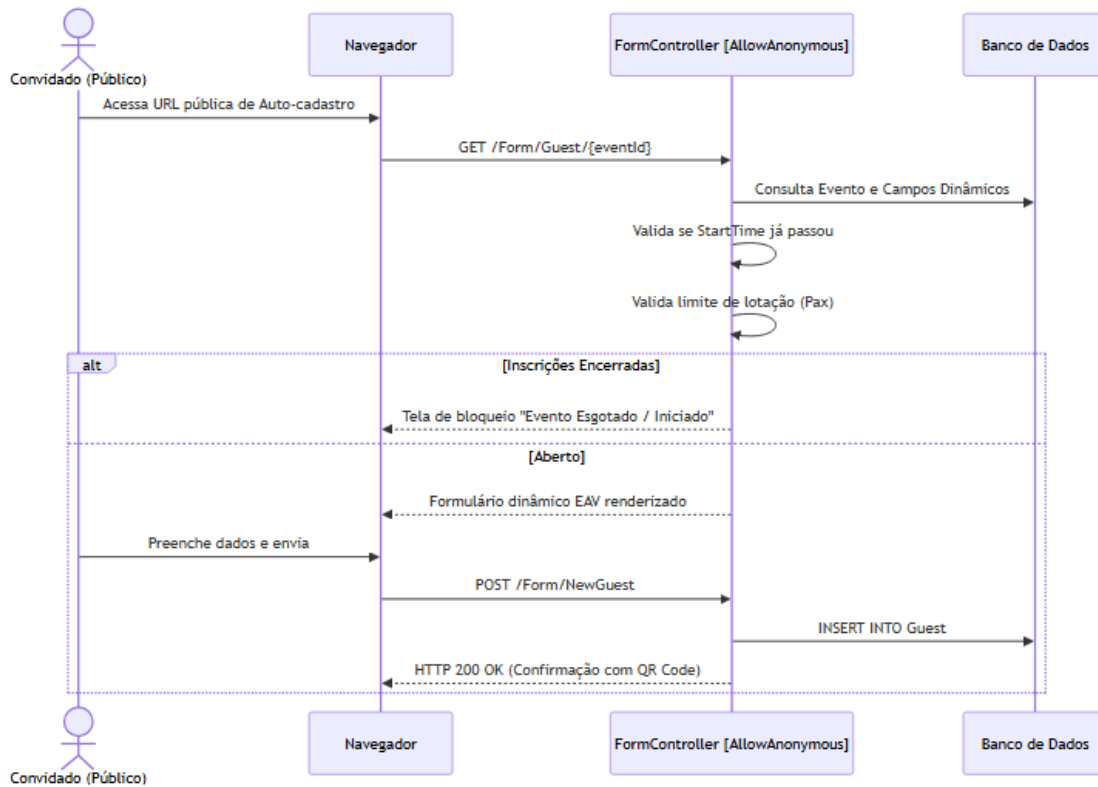


Figura 10 – Diagrama de Sequência: Check-in em Lote

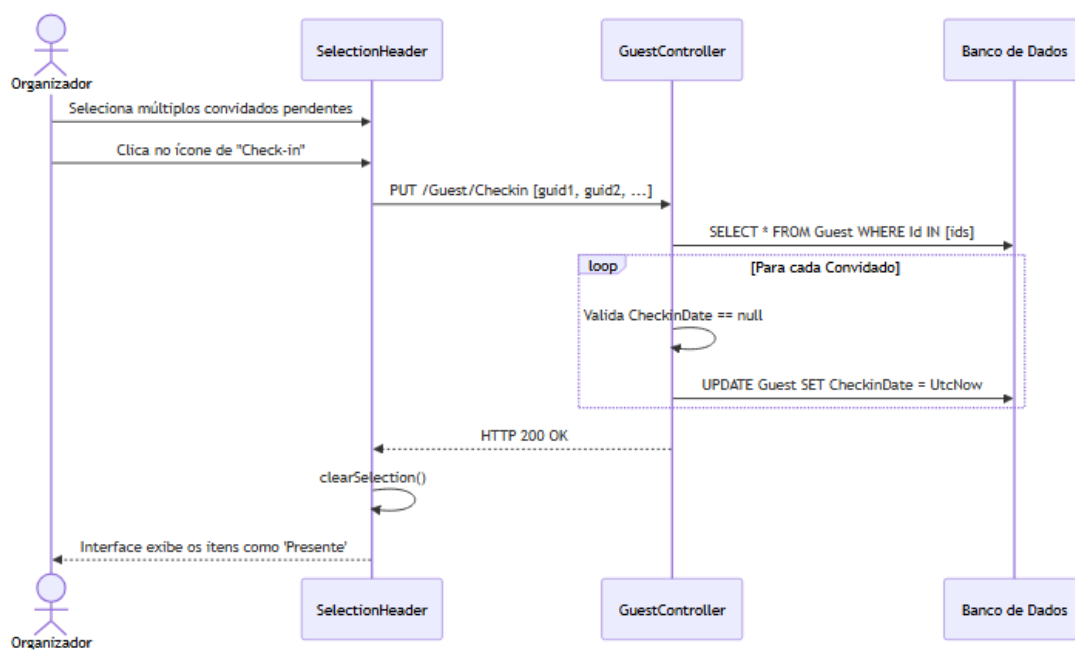
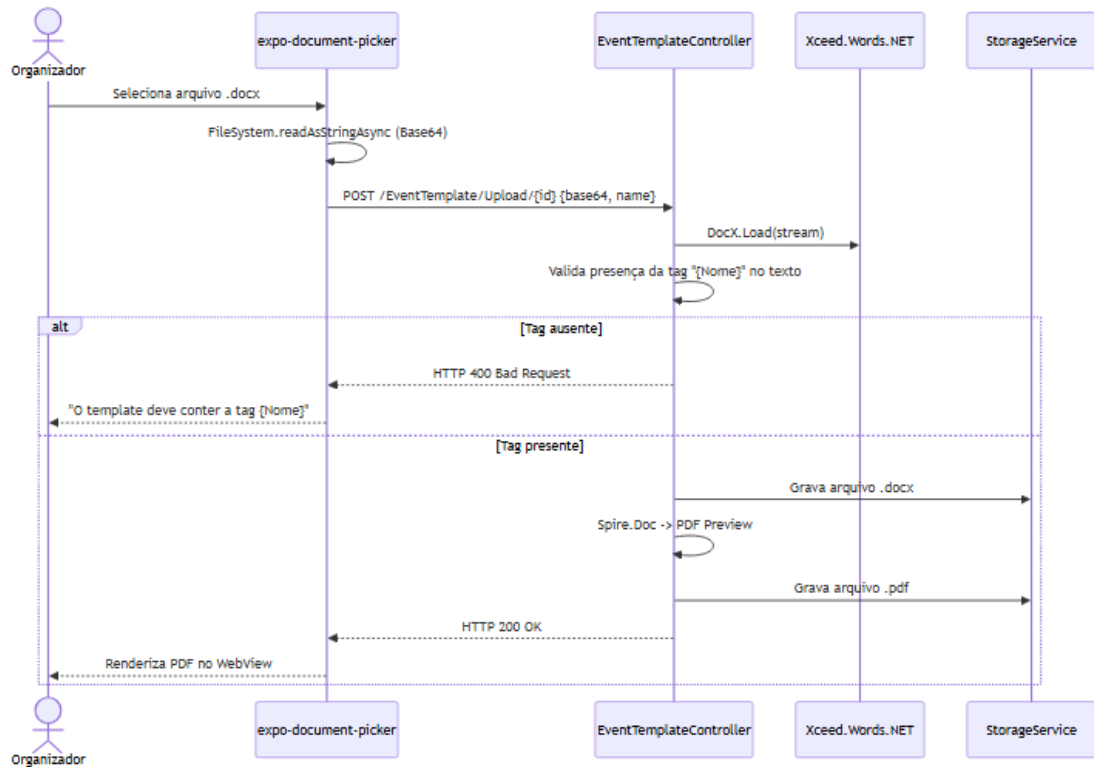
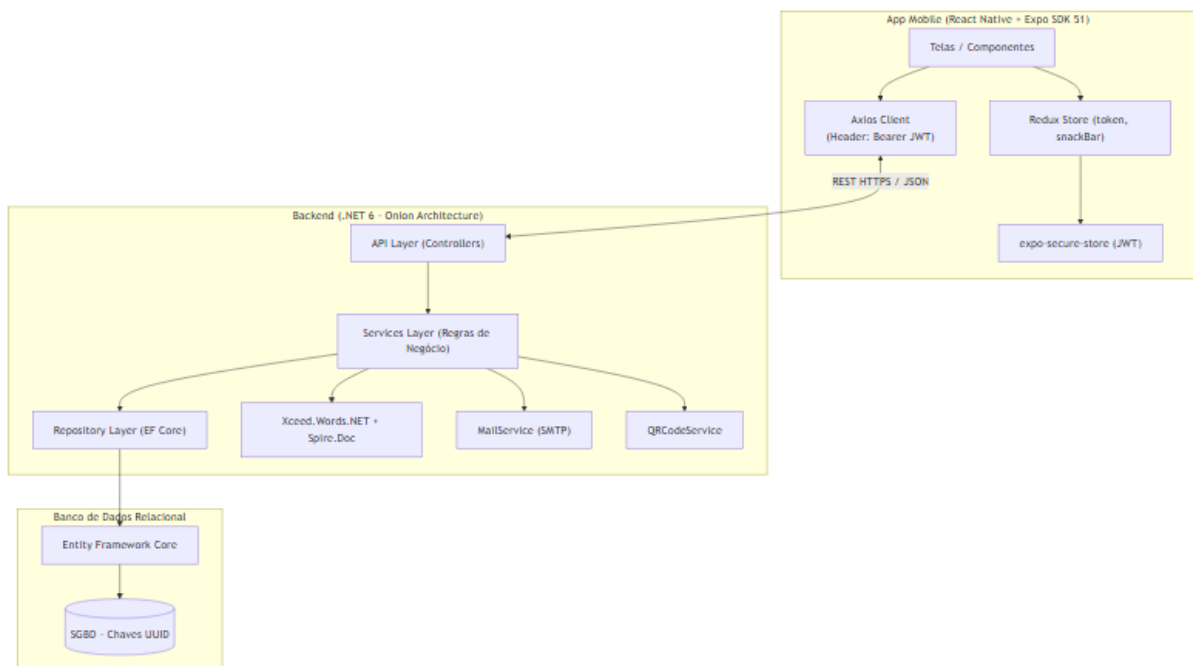


Figura 11 – Diagrama de Sequência: Upload de Template .docx



4.7 Visão Geral do Projeto do Sistema

Figura 12 – Arquitetura Distribuída: Cliente, Servidor e Banco de Dados



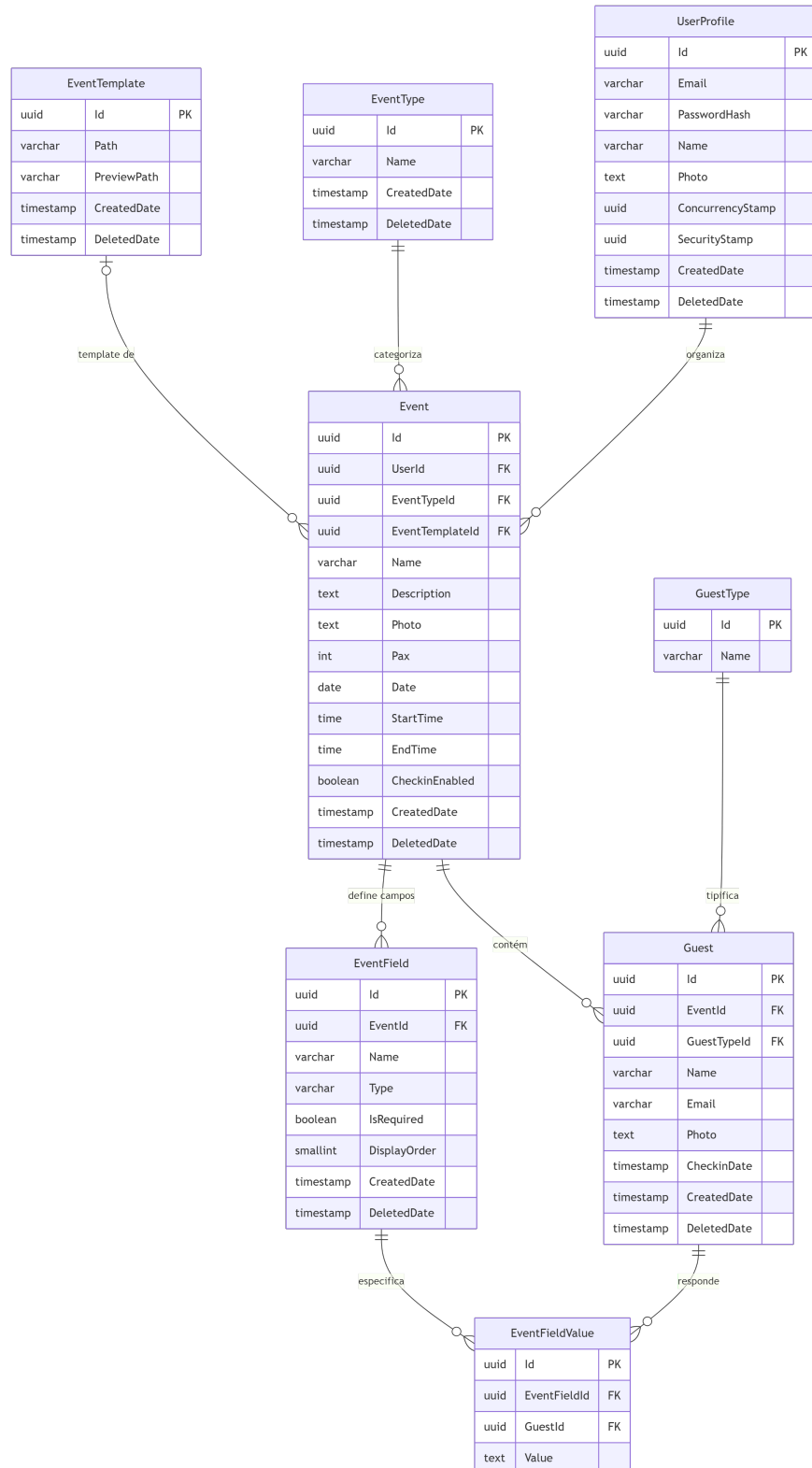
4.8 Visão do Banco de Dados

4.8.1 SGBD

O sistema utiliza o **Entity Framework Core** como ORM (*Object-Relational Mapper*), que abstrai as consultas SQL e permite o mapeamento direto entre as entidades C# e as tabelas relacionais. A adoção de chaves primárias **Guid** torna a aplicação agnóstica ao SGBD, sendo compatível com SQL Server, PostgreSQL e SQLite sem alteração de código no domínio.

4.8.2 MER - Modelo de Entidade Relacional

Figura 13 – Modelo de Entidade Relacional (MER)



4.9 Visão do Backend (API RESTful)

4.9.1 Tecnologias a serem usadas

- **ASP.NET Core 6 (C#):** Framework principal da API REST, responsável pelo roteamento, injeção de dependências e *middleware* de autenticação JWT.
- **Entity Framework Core:** ORM responsável pelo mapeamento objeto-relacional e execução de queries via LINQ.
- **Xceed.Words.NET (DocX):** Biblioteca para manipulação de documentos .docx em memória. Usada para carregar o template e realizar substituições via `StringReplaceTextOptions`.
- **Spire.Doc:** Biblioteca de conversão de documentos Word para PDF, operando integralmente via `MemoryStream` sem geração de arquivos temporários.
- **JWT Bearer Authentication:** Middleware [`Authorize`] que valida a assinatura HMAC-SHA256 do token em todas as rotas protegidas.

4.9.2 Projeto do Backend (Onion Architecture)

A API é estruturada em cinco projetos isolados que seguem a *Onion Architecture*:

- **Domain:** Contém entidades (`Event`, `Guest`, etc.), interfaces de serviços e repositórios, DTOs e exceções de negócio (`BusinessException`, `NotFoundException`, `ConflictException`). Sem nenhuma dependência externa.
- **Services:** Implementações das regras de negócio (validações, geração de QR Code, substituição de *placeholders*, envio de e-mail). Dependem apenas de interfaces do `Domain`.
- **Repository:** Implementações dos repositórios com EF Core. Aplicam o filtro de *Soft Delete* nas consultas.
- **Infra:** Implementações de serviços externos (armazenamento de arquivos, e-mail SMTP, hash de senhas).
- **API:** Camada de apresentação. Contém os *Controllers* e o *pipeline* de configuração do ASP.NET Core.

4.9.3 Endpoints da API

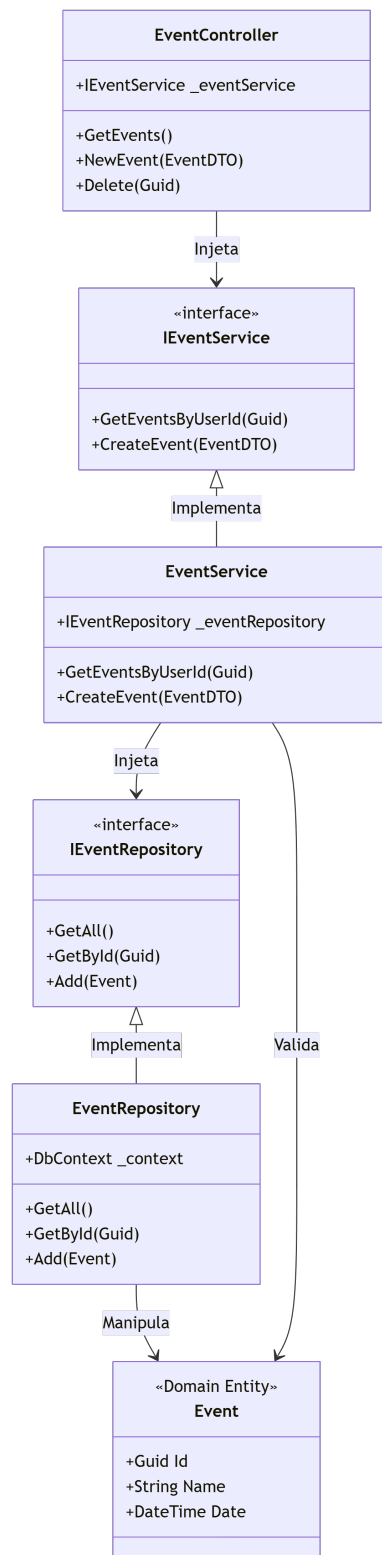
Tabela 7 – Mapeamento Completo de Endpoints da API Certify

Método	Rota	Descrição	Auth
POST	/Account/Login	Autentica e retorna JWT	Não
GET	/Event/GetEvents	Lista eventos do usuário logado	Sim
GET	/Event/GetEvent/{id}	Retorna evento com Guests e Fields	Sim
POST	/Event/NewEvent	Cria novo evento	Sim
DELETE	/Event/Delete/{id}	Soft delete do evento	Sim
PUT	/Event/CheckinEnabledMode	Altera o modo de check-in	Sim
GET	/Event/QRCode/{eventId}	Gera QR Code do link de auto check-in	Não
GET	/Event/QRCode/decode/{eventId}	Decodifica hash do evento	Não
GET	/Event/Certificado/Download/{id}	Gera ZIP com todos os PDFs	Sim
POST	/Event/Certificado/Send/{id}	Envia certificados por e-mail	Sim
GET	/Event/{id}/EventField	Lista campos EAV do evento	Sim
POST	/Event/EventField	Cria novo campo dinâmico	Sim
PUT	/Event/EventField/Reorder	Reordena campos (DTO)	Sim
PUT	/Event/{id}/EventField/Reorder	Reordena campos (array Guid)	Sim
DELETE	/EventField/{id}	Remove campo dinâmico	Sim
POST	/EventTemplate/Upload/{id}	Upload do template .docx	Sim
DELETE	/EventTemplate/Remove/{id}	Remove template do evento	Sim
GET	/EventType/GetEventTypes	Lista tipos de eventos	Sim
GET	/GuestType/GetGuestTypes	Lista tipos de convidados	Sim
POST	/Guest/NewGuest	Cadastra convidado (admin)	Sim
PUT	/Guest/Checkin/{id}	Check-in individual	Sim
PUT	/Guest/Uncheckin/{id}	Reverte check-in individual	Sim
PUT	/Guest/Checkin	Check-in em lote [array Guid]	Sim
PUT	/Guest/Uncheckin	Reverte check-in em lote	Sim
DELETE	/Guest/Delete/{id}	Soft delete de convidado	Sim
DELETE	/Guest/Delete	Soft delete em lote	Sim
POST	/Guest/Invitations/{eventId}	Envia convites por e-mail em lote	Sim
POST	/Guest/Certificates/{eventId}	Gera e envia certificados selecionados	Sim
GET	/Guest/QRCode/{guestId}	Gera QR Code do convidado (hash)	Não
GET	/Form/Guest/{eventId}	Formulário público de inscrição	Não
POST	/Form/NewGuest	Submete inscrição pública	Não
GET	/Form/Checkin/{eventId}	Formulário público de check-in	Não
GET	/Form/Guest/Search/{code}	Busca convidado por código de acesso	Não
PUT	/Form/Guest/Checkin/{code}	Auto check-in via código de acesso	Não

4.9.4 Estrutura de Componentes da API

Para demonstrar a aplicação prática da *Onion Architecture* através da Injeção de Dependências, a Figura 8 ilustra como a camada de *API Layer* (Controllers) interage com o *Services Layer* via interfaces puras. A inversão de dependência garante que o controlador desconheça a implementação concreta do repositório, garantindo desacoplamento.

Figura 14 – Estrutura do Componente Evento (Onion Architecture)



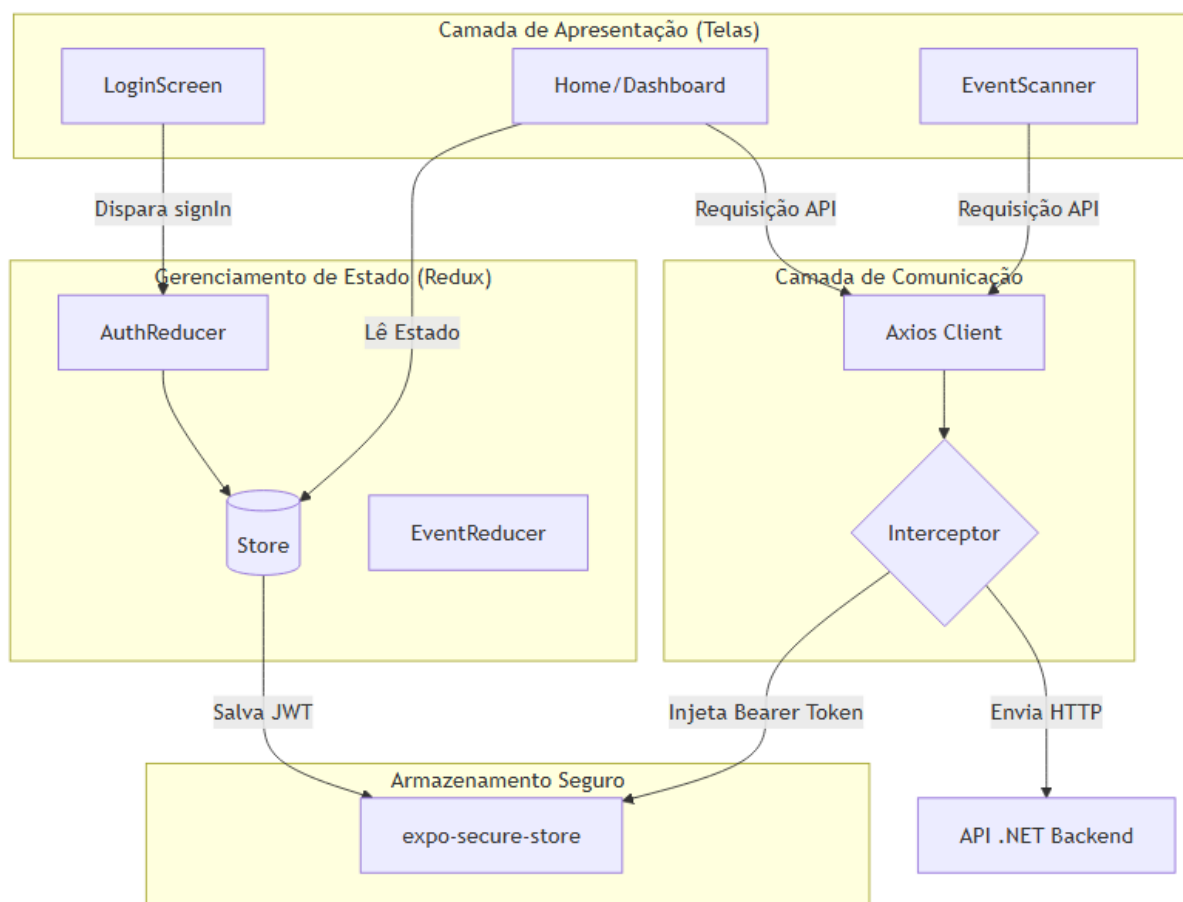
4.10 Visão do Frontend (App Mobile)

Assim como o backend, o lado do cliente exige um gerenciamento arquitetural para manter as responsabilidades separadas de forma sustentável, uma vez que o App Mobile é responsável por entregar a interface do organizador e o fluxo contínuo do scanner de QRCode para check-ins.

O ecossistema do frontend utiliza o **React Native** juntamente com o **Expo SDK**. A comunicação é realizada por meio do cliente HTTP *Axios*, que utiliza interceptadores (*interceptors*) para anexar automaticamente o token *Bearer* nas requisições. O **Redux** é empregado como Single Source of Truth (Fonte Única da Verdade), gerenciando não apenas a sessão do usuário de forma reativa, mas também o cache de eventos local, a fim de diminuir o *round-trip* ao servidor, essencial em ambientes de rede lenta durante congressos e eventos presenciais.

A segurança no cliente é implementada salvando a chave primária JWT (JSON Web Token) no `expo-secure-store`, evitando acesso à memória cache e restringindo-o apenas via keystore nativa do SO.

Figura 15 – Fluxo da Camada de Frontend (App Mobile)



5 Apresentação do Sistema

5.1 Tecnologias a serem usadas no Frontend

O aplicativo foi desenvolvido em **React Native 0.74** com o runtime do **Expo SDK 51**, utilizando o modelo de componentes funcionais e API de Hooks do React. As principais bibliotecas adotadas são:

- **@react-navigation/native + createNativeStackNavigator:** Controle da hierarquia de telas (pilhas pública e privada) e navegação programática entre rotas.
- **Redux Toolkit:** Gerência global de estado com dois *slices*: `token.js` (sessão JWT) e `snackBar.js` (notificações de feedback global).
- **expo-secure-store:** Armazenamento criptografado do token JWT no cofre nativo do sistema operacional (*Keychain* no iOS, *KeyStore* no Android).
- **expo-camera (CameraView):** Captura de câmera nativa para leitura de QR Code com validação por `boundingBox`.
- **react-native-gesture-handler (Swipeable):** Gestos de deslizamento horizontal em cards da lista de eventos e convidados.
- **react-native-draggable-flatlist:** Lista com suporte a reordenação por arrastar-e-soltar (*Drag 'n Drop*) a 60fps.
- **react-native-paper:** Componentes visuais baseados em *Material Design* (inputs, dialogs, FAB).
- **expo-font (Poppins):** Família tipográfica carregada assincronamente na inicialização (Regular, Bold, Black, ExtraBold).
- **expo-image-picker:** Seleção e compressão de fotos da galeria ou câmera.
- **expo-document-picker + expo-file-system:** Seleção de arquivos `.docx` e leitura em Base64 para upload do template de certificado.
- **react-native-webview:** Renderização do PDF de pré-visualização do certificado diretamente no aplicativo.
- **Axios:** Cliente HTTP com interceptors que injetam automaticamente o cabeçalho `Authorization: Bearer {token}` em todas as requisições autenticadas.

- **@fortawesome/react-native-fontawesome:** Ícones vetoriais (lixeira, chevron, check, refresh, etc.).
- **date-fns:** Utilitário de formatação de datas no frontend (ex: `format(date, "dd/MM/yyyy")`).

5.2 Projeto das Interfaces

A seguir são apresentadas as interfaces do sistema Certify, com descrições baseadas na análise direta dos arquivos-fonte de cada tela. As imagens deverão ser substituídas pelas capturas de tela reais do aplicativo durante a etapa de finalização do documento.

Figura 16 – Tela de Login (`login.js`)

[IMAGEM: Captura da tela de Login. Exibir: tela com fundo estilizado (splash), ícone de QR Code centralizado no topo, campo de E-mail com validação de formato em tempo real, campo de Senha com botão "olho" para alternar visibilidade, botão "Entrar" com indicador de loading (`ButtonLoading`) e exibição de snackbar de erro em caso de credenciais inválidas.]

A tela de Login é o ponto de entrada do ecossistema privado. O aplicativo executa validações locais de formato antes de qualquer chamada à API. Em caso de sucesso, o token JWT é persistido via `expo-secure-store` e a *Redux Store* é atualizada com `dispatch(signIn(token))`, o que dispara a remontagem reativa da pilha de navegação para a pilha privada (Home).

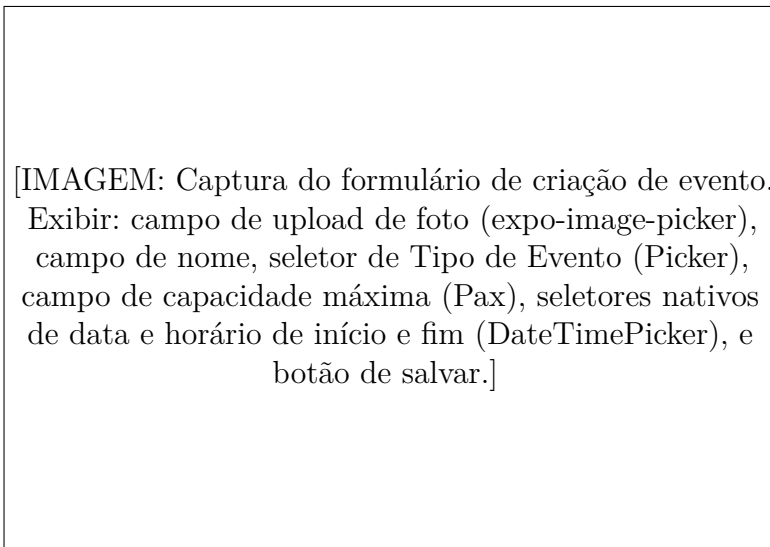
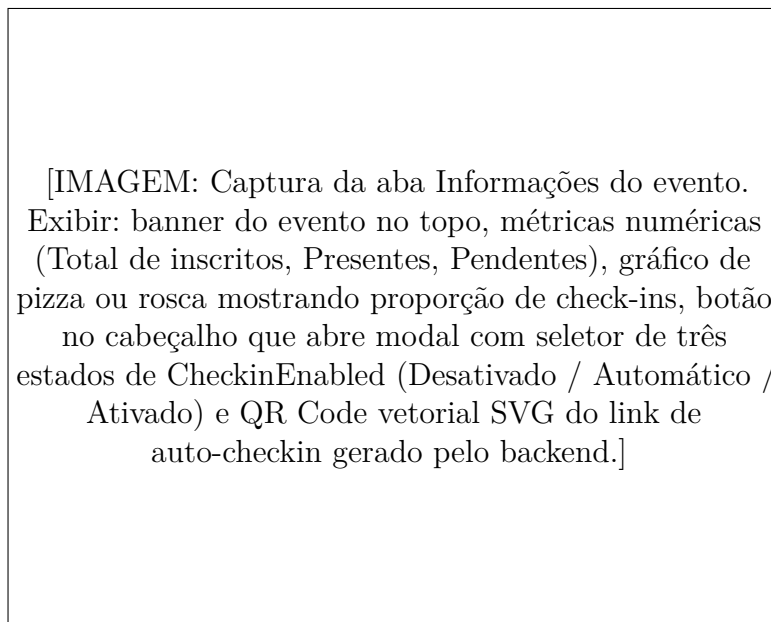
Figura 17 – Tela de Criar Conta (`register.js`)

[IMAGEM: Captura da tela de Registro. Exibir: campos para Nome completo, E-mail e Senha com botão "Criar Conta". Design consistente com a tela de Login.]

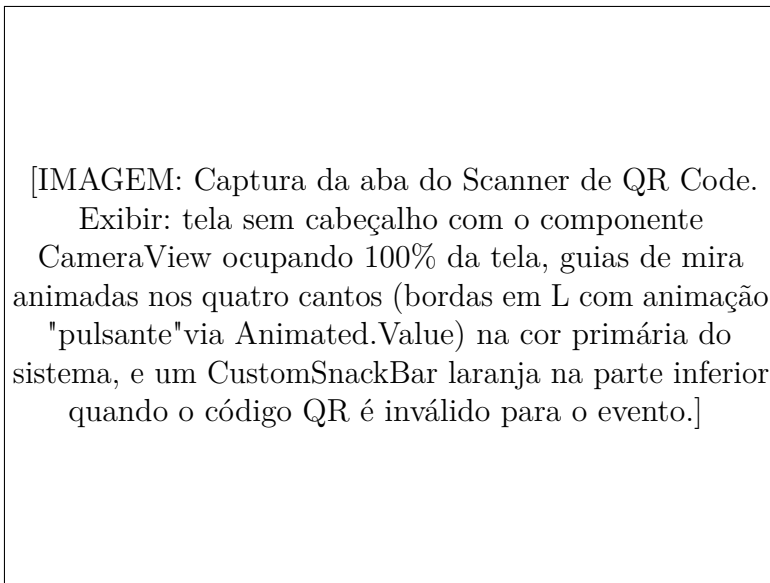
Figura 18 – Tela Inicial — Lista de Eventos (`home.js`)

[IMAGEM: Captura da Home. Exibir: cabeçalho com botão de logout e ícone de refresh, `SegmentedControl` com abas "Todos / Criado / Ativo / Fim" para filtro em memória, `FlatList` com cards de eventos (foto, nome, tipo e data), card sendo deslizado para a esquerda expondo botão vermelho de lixeira (`Swipeable`), botão flutuante (+) verde no canto inferior direito.]

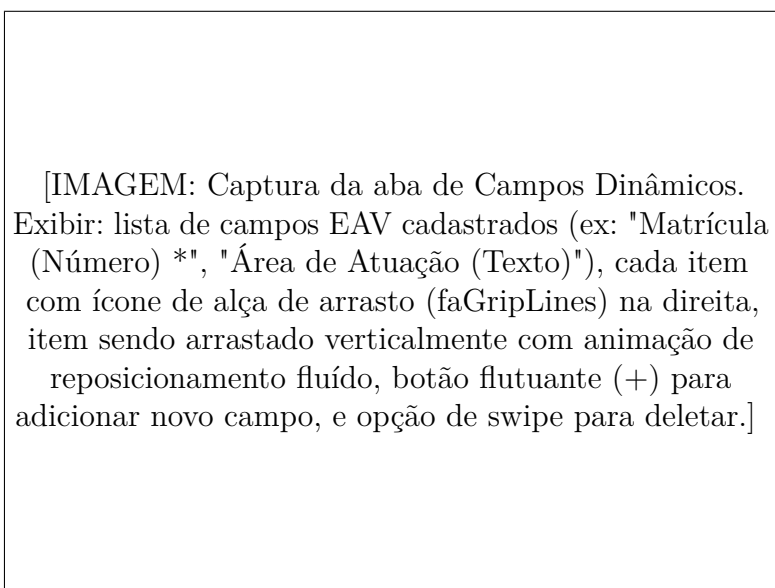
A tela Home lista exclusivamente os eventos do organizador autenticado, cujo UUID é extraído do token JWT pelo `UserContextService` no backend. Os filtros de status são aplicados em memória no frontend (`filter()` sobre o array `events`), evitando chamadas redundantes ao servidor. O gesto de *swipe* para a esquerda é gerenciado pelo componente `Swipeable` da biblioteca *react-native-gesture-handler*, que expõe o botão de exclusão com abertura de modal de confirmação (`ConfirmAlert`).

Figura 19 – Criação de Novo Evento (`newEvent.js`)Figura 20 – Aba de Informações do Evento (`eventTab.js`)

A aba de informações consome o endpoint `GET /Event/GetEvent/{id}` que retorna o evento com os dados dos convidados embutidos. O organizador pode alterar o modo de check-in via modal, que aciona o endpoint `PUT /Event/CheckinEnabledMode`, persistindo o valor `null` (Automático), `true` (Ativado) ou `false` (Desativado) no campo `CheckinEnabled` da tabela `Event`.

Figura 21 – Leitor de QR Code (`eventScanner.js`)

A tela de scanner maximiza a área útil do visor ao remover o cabeçalho padrão. A animação das bordas de mira é implementada com `Animated.loop + Animated.sequence`, expandindo e contraindo suavemente os cantos entre 1000ms cada ciclo via `useNativeDriver: false`. A leitura é considerada válida apenas se o `boundingBox` do código QR detectado estiver dentro dos limites do quadro de mira (`scannerBounds`), evitando leituras acidentais de códigos fora do enquadramento.

Figura 22 – Gerenciamento de Campos Dinâmicos (`eventFields.js`)

A reordenação dos campos utiliza o componente `DraggableFlatList` integrado ao

`react-native-reanimated`. O evento `onDragEnd` captura o array reordenado e dispara uma requisição `PUT /Event/{id}/EventField/Reorder` enviando o vetor de GUIDs na nova ordem; o backend itera o array e atualiza sequencialmente o `DisplayOrder` de cada `EventField`.

Figura 23 – Criação de Novo Campo Dinâmico (`newField.js`)

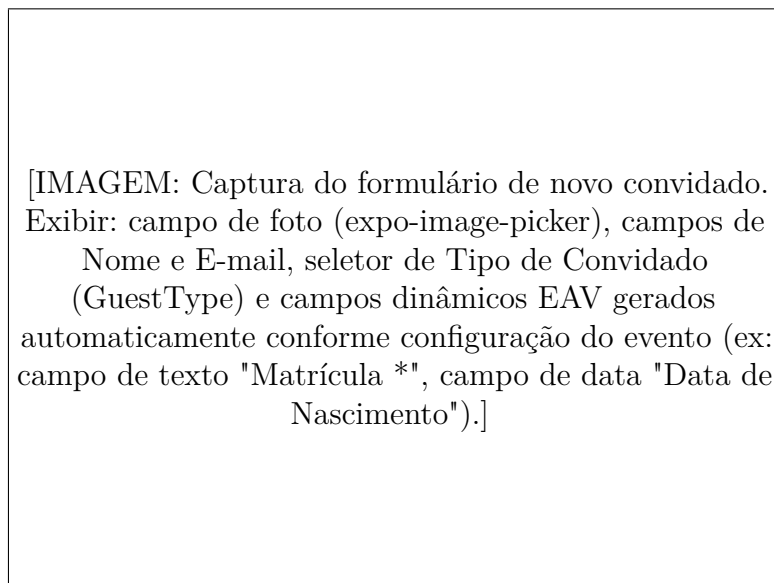
[IMAGEM: Captura do formulário de criação de campo. Exibir: input de Nome do campo, seletor segmentado de Tipo ("Texto", "Número", "Data"), seletor segmentado de obrigatoriedade ("Opcional" / "Obrigatório") e botão "Salvar".]

Figura 24 – Lista de Convidados e Seleção em Massa (`guestsTab.js`)

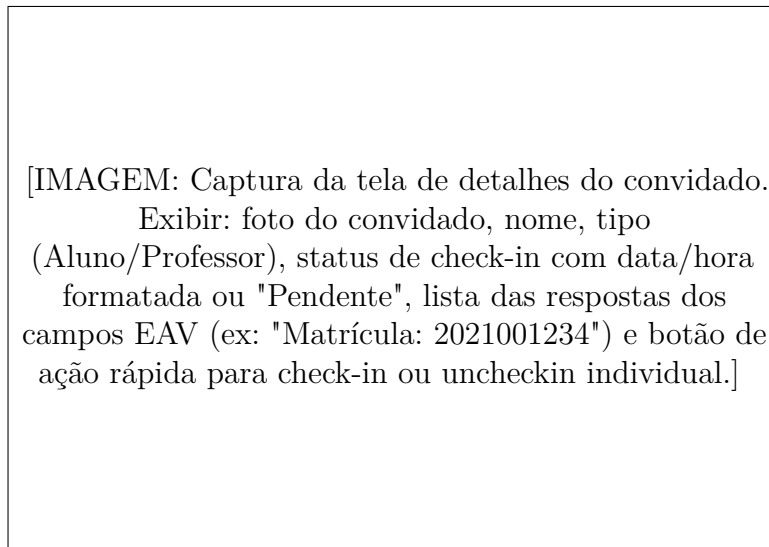
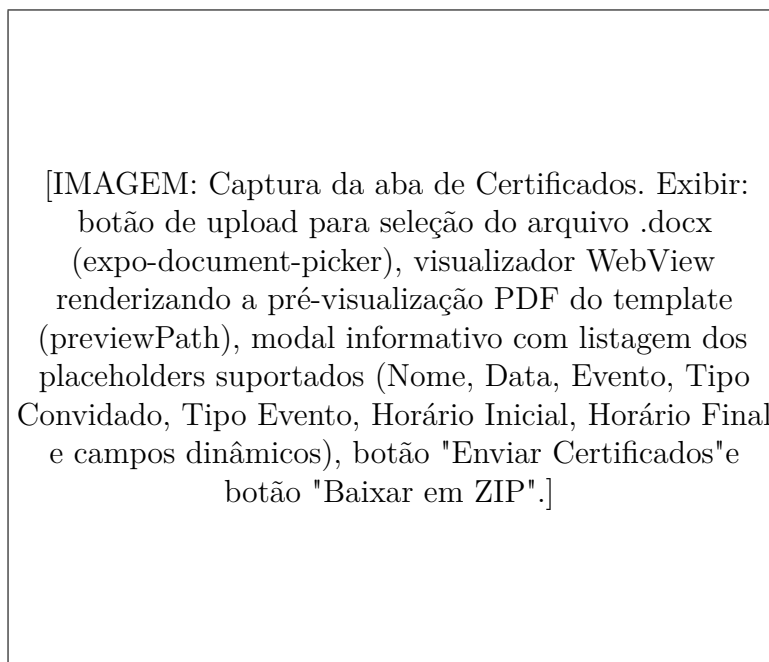
[IMAGEM: Captura da aba de Convidados com modo de seleção em massa ativado. Exibir: `SelectionHeader` na cor primária mostrando a contagem de selecionados (ex: "3") com ícones de lixeira, check-in e menu de três pontos (`faEllipsisV`); dois `SegmentedControls` sobrepostos (`Todos/Pendente/Checkin` e `Todos/Aluno/Professor`); cards de convidados com foto, nome, tipo e status de check-in (data ou "Pendente"); checkboxes visuais nos itens selecionados substituindo a foto pelo ícone `faCheck`; item sendo deslizado (swipe) expondo botões coloridos de exclusão (vermelho) e check-in (verde) ou uncheckin (amarelo).]

A aba de convidados é a mais complexa do sistema. O modo de seleção em massa é ativado com `onLongPress` (400ms), que adiciona o item ao array `selectedItems` e substitui o cabeçalho padrão pelo `SelectionHeader`. Os dois filtros `SegmentedControl` (status e tipo) operam em memória sobre o array `guests` sem requisições adicionais. O menu de contexto (`PopupMenu`) acionado pelo botão de três pontos oferece as ações em lote: *Enviar certificado* (para selecionados com check-in) e *Enviar convite* (para selecionados sem check-in), além de exclusão e inversão de check-in.

Figura 25 – Cadastro Manual de Convidado (`newGuest.js`)



Os campos EAV são buscados via `GET /Event/{id}/EventField` e renderizados dinamicamente: `Type == "Text"` gera um `TextInput` padrão; `Type == "Number"` gera `keyboardType="numeric"`; `Type == "Date"` renderiza um `DateTimePicker` nativo. Campos com `IsRequired = true` bloqueiam o envio do formulário se estiverem vazios.

Figura 26 – Detalhes do Convidado (`guest.js`)Figura 27 – Aba de Certificado e Template (`templateTab.js`)

O upload do template lê o arquivo selecionado como Base64 via `FileSystem.readAsStringAsync(ur { encoding: EncodingType.Base64 })` e envia para `POST /EventTemplate/Upload/{eventId}`. O backend valida o tipo MIME, o tamanho máximo (10 MB) e a presença obrigatória do *placeholder* `{Nome}` utilizando a biblioteca `Xceed.Words.NET` antes de aceitar o arquivo. Após aprovação, ambos o `.docx` original e a pré-visualização `.pdf` são gravados no `StorageService` e seus caminhos persistidos na tabela `EventTemplate`.

6 Considerações Finais

6.1 Conclusão

O presente trabalho apresentou a arquitetura e implementação ponta-a-ponta do Certify, um sistema de gestão de eventos robusto concebido para sanar ineficiências na emissão de certificados, controle temporal de fluxo e customizações em formulários de inscrição.

A adoção de tecnologias de performance, como o framework .NET 6 com a Onion Architecture nas lógicas severas do motor de credenciais e processamento massivo de arquivos PDF na memória (via Streams assíncronas), demonstrou que as APIs RESTful adequadamente configuradas evitam degradação física da infraestrutura de servidores.

No pilar de interface e credenciamento móvel, o React Native provou-se flexível para controlar lógicas densas de câmera óptica e arrasto sensível na tela, aliadas ao fechamento rigoroso de sessão com a encriptação direta nativa (SecureStore/Keychain). Além disso, a premissa de um banco de dados fundamentado no mecanismo UUID associado ao conceito *Entity-Attribute-Value* cumpriu seu propósito, dotando a ferramenta de elasticidade sem sobrecarga do SGBD. A conclusão técnica deste desenvolvimento constata que a substituição dos métodos obsoletos de entrada e edição analógica nas bilheterias pela criptografia vetorial de QR Codes eleva drasticamente o rendimento logístico da organização.

6.2 Implementações Futuras

Como a base da plataforma encontra-se modularizada e devidamente desacoplada pelo front e backend, atualizações de características podem ser integradas facilmente ao ecossistema. Dentre as sugestões imediatas para melhorias iterativas do sistema Certify, citam-se:

1. **Geolocalização para Check-in Público:** Implementar maior segurança na validação do convidado, exigindo que ele compartilhe a localização do celular no momento do auto-checkin. Isso evita fraudes e garante que o participante só consiga confirmar presença se estiver fisicamente no perímetro do evento.
2. **Templates Pré-definidos no App:** Disponibilizar uma galeria de layouts de certificados diretamente no aplicativo, permitindo que o organizador crie, edite

e personalize o visual do documento sem depender do upload de arquivos ‘.docx’ externos.

3. **Abordagem *Offline-First* no Frontend:** Implementação de bancos de dados nativos na memória móvel (como o Realm ou SQLite nativo no Expo). Se houver perda total de acesso à internet durante as palestras, os check-ins serão enfileirados localmente e enviados à nuvem apenas quando o organizador restaurar a conexão, sem interromper as filas.
4. **Dashboard de BI (Business Intelligence):** Adicionar um módulo complementar para elaboração visual de métricas de retenção (horários médios de saída dos eventos, cruzamentos demográficos a partir dos campos EAV e estatísticas operacionais de evasão).